

Introdução à Complexidade Computacional

Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

O Que é um Algoritmo?

Um **algoritmo** é qualquer procedimento computacional bem definido que recebe algum valor, ou conjunto de valores, como **entrada** e produz algum valor, ou conjunto de valores como **saída**

Problema de Ordenação

- Podemos definir formalmente um **problema de ordenação**:
 - **Entrada:** Uma sequência de n números $\langle a_1, a_2, \dots, a_n \rangle$
 - **Saída:** Uma permutação $\langle a'_1, a'_2, \dots, a'_n \rangle$ da sequência de entrada tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$
- Por exemplo, dado a sequência de entrada:
 $\langle 31, 41, 59, 26, 41, 58 \rangle$
- Um algoritmo de ordenação retorna como saída a sequência
 $\langle 26, 31, 41, 41, 58, 59 \rangle$

Eficiência

- Algoritmos diferentes criados para resolver o mesmo problema muitas vezes são muito diferentes em termos de eficiência
 - Diferenças podem ser muito mais significativas que as diferenças relativas a hardware e software
- Sejam dois algoritmos de ordenação:
 - **Ordenação por inserção** $\Rightarrow$ tempo $\approx c_1 n^2$ para ordenar n itens, onde c_1 é uma constante que não depende de n
 - **Ordenação por intercalação** $\Rightarrow$ tempo $\approx c_2 n \lg n$, onde $\lg n$ representa $\log_2 n$ e c_2 é outra constante que também não depende de n

A x B

- Computador mais rápido (**Computador A**) que executa a ordenação por inserção *vs* computador mais lento (**computador B**) que executa a ordenação por intercalação
 - Ordenar 1 milhão de números
 - $c_1 = 2, c_2 = 50$

$$A = \frac{2 \cdot (10^6)^2 \text{ instruções}}{10^9 \text{ instruções/segundo}} = 2000 \text{ segundos}$$

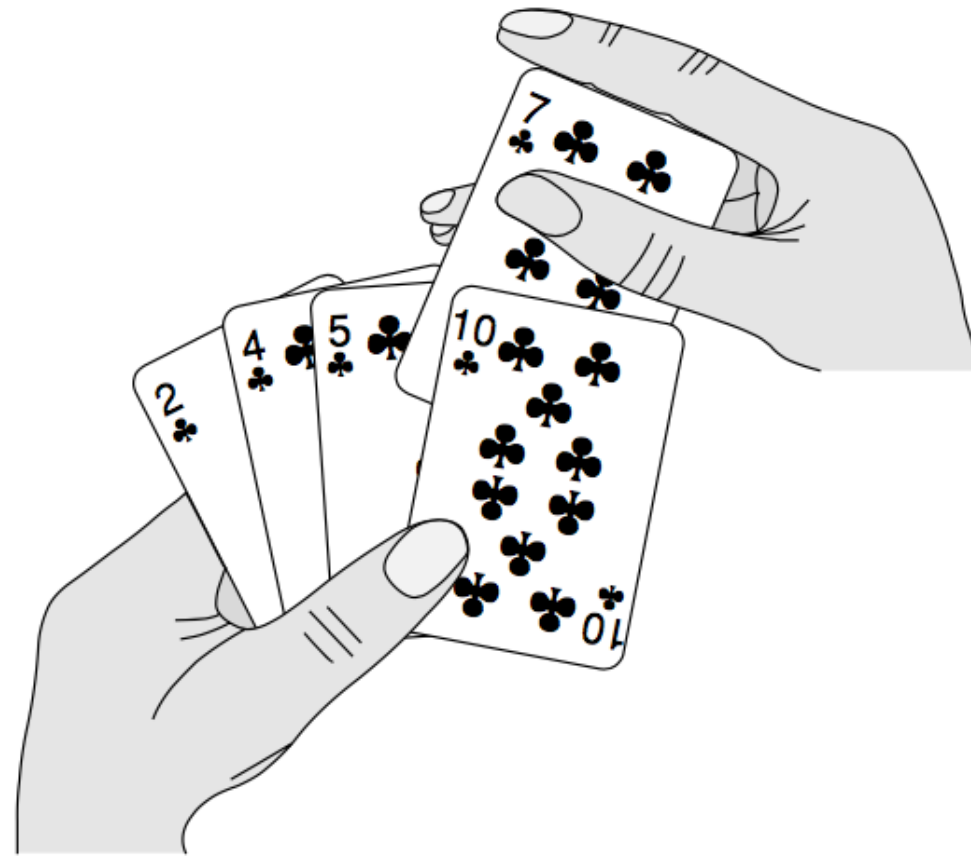
$$B = \frac{50 \cdot 10^6 \lg 10^6 \text{ instruções}}{10^7 \text{ instruções/segundo}} \approx 100 \text{ segundos}$$

A x B (Cont.)

- A vantagem da ordenação por intercalação é ainda mais evidente quando ordenamos **10** milhões de números
 - Ordenação por inserção leva **2,3** dias
 - Ordenação por intercalação leva **20** minutos

Ordenação por Inserção

Ordenação por inserção, ou *insertion sort*, é um algoritmo de ordenação.



Ordenando uma mão de cartas usando ordenação por inserção.

Ordenação por Inserção: C

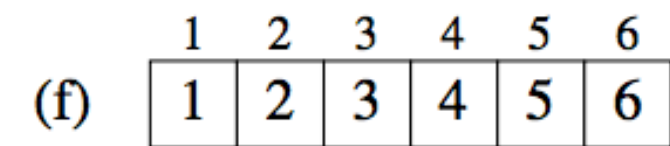
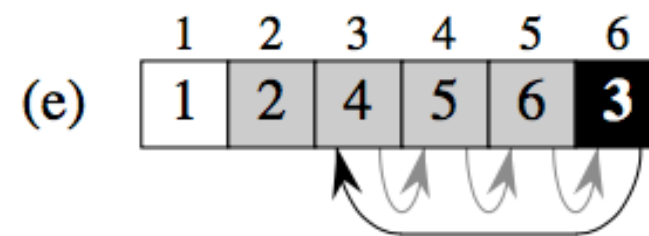
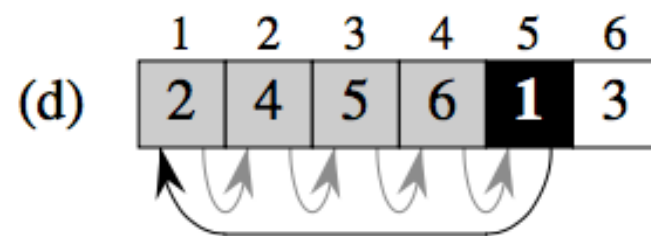
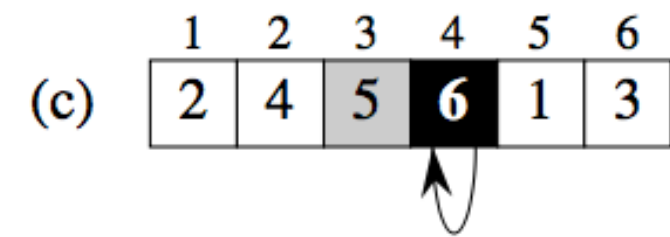
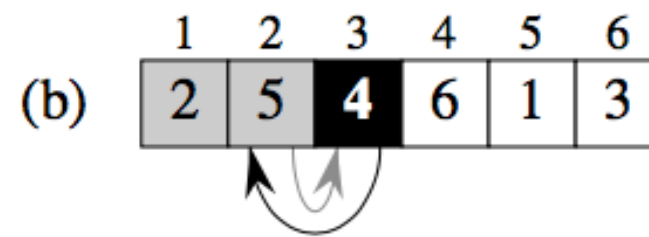
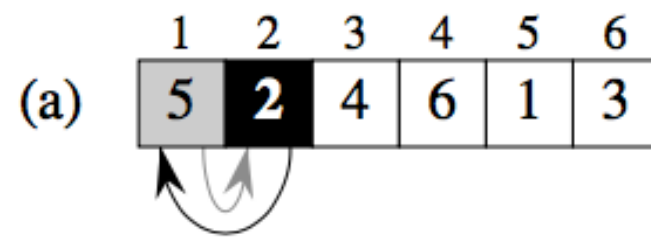
```
1. void ordenacao_por_insercao(int v[], int n) {
2.     int i, j, chave;
3.
4.     for(j = 1; j < n; j++) {
5.         chave = v[j];
6.         i = j - 1;
7.         // insere A[j] na sequência ordenada A[0..j-1].
8.         while (i >= 0 && v[i] > chave) {
9.             v[i+1] = v[i];
10.            i = i - 1;
11.        }
12.        v[i+1] = chave;
13.    }
14.}
```


Ordenação por Inserção: Pseudocódigo

INSERTION-SORT(A)

1. **for** $j \leftarrow 2$ **to** $length[A]$
2. **do** $key \leftarrow A[j]$
3. $\triangleright$ Insere $A[j]$ na sequência ordenada $A[1..j - 1]$.
4. $i \leftarrow j - 1$
5. **while** $i > 0$ and $A[i] > key$
6. **do** $A[i + 1] \leftarrow A[i]$
7. $i \leftarrow i - 1$
8. $A[i + 1] \leftarrow key$

Ordenação por Inserção: Ideia



Estudo de Algoritmos

- Dois aspectos estudo de algoritmos:
 - **Correção** verifica exatidão do método, o que é realizado através de uma prova matemática
 - **Análise** visa à obtenção de parâmetros para avaliar eficiência do algoritmo em termos de tempo de execução e memória ocupada
 - Estamos interessados na análise!!

Análise Ordenação por Inserção

Tempo Despendido Pelo Algoritmo

- **Tempo despendido** pelo algoritmo depende da entrada
 - Ordenar mil números demora mais que ordenar três números
 - Além disso, pode-se levar diferentes quantidades de tempo diferentes para entradas do mesmo tamanho
- Em geral, o tempo gasto por um algoritmo cresce com o tamanho da entrada $\Rightarrow$ assim, é tradicional descrever o tempo de execução de um programa como uma **função do tamanho da entrada**

Tamanho da Entrada Depende do Problema

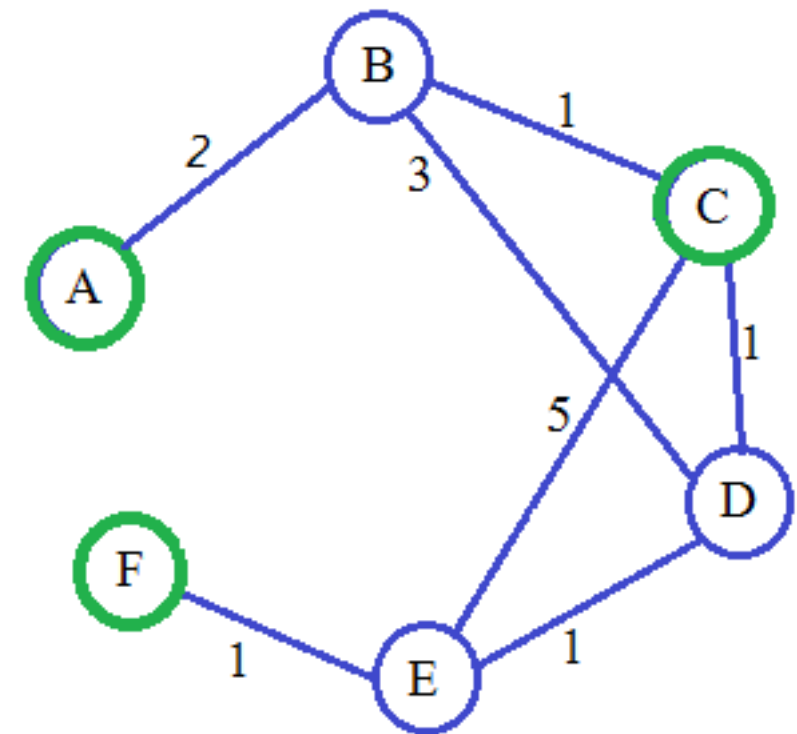
Array :

3	8	1	0	5	-2	32
---	---	---	---	---	----	----

Indices:

0 1 2 3 4 5 6

Em um algoritmo de ordenação, o tempo do algoritmo em geral é proporcional a quantidade de elementos que se deseja ordenar.



Em um algoritmo de busca em um grafo, o tempo do algoritmo pode ser proporcional ao número de vértices ou número de arestas.

Tempo de Execução

- É o número de operações primitivas ou **“passos” executados**
- Definiremos a noção de passo como sendo **independente de máquina** quanto possível
- Definiremos que uma quantidade de tempo constante é necessária para executar cada linha do pseudocódigo

Ordenação por Inserção: Tempo de Execução

INSERTION-SORT(A)

1. **for** $j \leftarrow 2$ **to** $length[A]$

2. **do** $key \leftarrow A[j]$

3. $\triangleright$ Insere $A[j]$ na sequência ordenada $A[1..j - 1]$.

4. $i \leftarrow j - 1$

5. **while** $i > 0$ and $A[i] > key$

6. **do** $A[i + 1] \leftarrow A[i]$

7. $i \leftarrow i - 1$

8. $A[i + 1] \leftarrow key$

Linha	Custo	Vezez
1	c_1	n
2	c_2	$n-1$
3	0	$n-1$
4	c_4	$n-1$
5	c_5	$\sum_{j=2}^n t_j$
6	c_6	$\sum_{j=2}^n (t_j - 1)$
7	c_7	$\sum_{j=2}^n (t_j - 1)$
8	c_8	$n-1$

Como Estimar Tempo de Execução?

- **Tempo de execução** é a soma de todos os tempos de cada instrução
- Podemos estimar o tempo de execução do algoritmo ordenação por inserção como:

$$\begin{aligned} T(n) = & c_1n + c_2(n - 1) \\ & + c_4(n - 1) + c_5\sum_{j=2}^n t_j \\ & + c_6\sum_{j=2}^n (t_j - 1) + c_7\sum_{j=2}^n (t_j - 1) \\ & + c_8(n - 1). \end{aligned}$$

Linha	Custo	Veze
1	c_1	n
2	c_2	$n-1$
3	0	$n-1$
4	c_4	$n-1$
5	c_5	$\sum_{j=2}^n t_j$
6	c_6	$\sum_{j=2}^n (t_j - 1)$
7	c_7	$\sum_{j=2}^n (t_j - 1)$
8	c_8	$n-1$

Entrada

- Mesmo para entradas do mesmo tamanho, o tempo de execução do algoritmo pode depende de qual entrada
- O **melhor caso** ocorre se o vetor já está ordenado
 - Para cada $j = 2, 3, \dots, n$, nós encontramos que $A[i] \leq chave \dots$

Pior, Melhor e Médio

- Seja um algoritmo A , $\{E_1, \dots, E_m\}$, o conjunto de todas as entradas possíveis de A . Denote por t_i , o número de passos efetuados por A , quando a entrada for E_i . Definem-se
 - **Complexidade do pior caso** $= \max_{E_i \in E} \{t_i\}$
 - **Complexidade do melhor caso** $= \min_{E_i \in E} \{t_i\}$
 - **Complexidade do caso médio** $\sum_{1 \leq i \leq m} p_i t_i$, onde p_i é a probabilidade de ocorrência da entrada E_i

Melhor Caso

- Para cada $j = 2, 3, \dots, n$, nós temos que $A[i] \leq chave$ na linha 5, quando i tem valor inicial de $j - 1$
- Assim, $t_j = 1$, para $j = 2, 3, \dots, n$ e o melhor caso é:
$$T(n) = c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1)$$
$$= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8)$$
- Este tempo de execução pode ser expresso como $an + b$ para constantes a e b que dependem dos custos c_i
- Assim, dizemos que é uma **função linear** de n

Pior Caso

- Ocorre se vetor estiver ordenado na ordem contrária
- Devemos comparar cada elemento de $A[j]$ com cada elemento do subarranjo ordenado inteiro, $A[1..j-1]$, e então $t_j = j$ para $j = 2, 3, \dots, n$
- Note que:

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

$$\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

Pior Caso (Cont.)

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\ &\quad + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8). \end{aligned}$$

- Temos $an^2 + bn + c$ para constantes a , b e c que, novamente, dependem dos custos de c_i
- Dizemos que esta é uma **função quadrática** de n

Por que Pior Caso?

- De maneira geral estaremos interessados no **pior caso**:
 - É o limite superior do tempo de execução;
 - Ocorre com bastante frequência;
 - O caso médio é frequentemente tão ruim quanto o pior caso.

Abstrações na Análise

- Adotamos algumas abstrações para simplificar a nossa análise do procedimento **ordenação por inserção**:
 - Ignoramos o custo real de cada instrução, por meio do uso de constantes c_i para representar estes custos
 - Então, observamos que até mesmo essas constantes nos dão mais detalhes do que realmente necessitamos
 - Expressamos o tempo de execução no pior caso é $an^2 + bn + c$ para algumas constantes a , b e c que dependem do custo de instrução c_i
 - Desse modo, ignoramos não apenas os custos reais de instrução, mas também os custos abstratos c_i

Ordem de Crescimento

- Agora, faremos mais uma abstração simplificadora
- É a **taxa de crescimento**, ou **ordem de crescimento**, do tempo de execução que realmente nos interessa
- Consideramos apenas no termo dominante da fórmula (e.g., an^2)
 - Termos de ordem mais baixa são relativamente insignificantes para grandes valores de n
- Também ignoramos o coeficiente constante do termo inicial
 - Fatores constantes são menos significativos que a taxa de crescimento na determinação da eficiência computacional para grandes entradas
- No caso de ordenação por inserção, quando ignoramos os termos de ordem mais baixa e o coeficiente constante do termo inicial, resta apenas o fator de n^2 do termo inicial

Notação Θ (Theta)

- Podemos escrever que o algoritmo **ordenação por inserção**, tem o tempo de execução no pior caso de $\Theta(n^2)$ (“**teta de n ao quadrado**”)
- Consideraremos um algoritmo ser mais **eficiente** que outro se o tempo de execução no pior caso tem uma ordem de crescimento menor
 - Algoritmo $\Theta(n^2)$ é mais eficiente que um algoritmo $\Theta(n^3)$
 - Devido a constantes (aditivas e multiplicativas), esta avaliação pode ser falha para pequenas entradas
 - Porém para grandes entradas é válida

Comparativo Tempo de Execução

We assume a 30 day month and 365 day year.

	1 Second	1 Minute	1 Hour	1 Day	1 Month	1 Year	1 Century
$\lg n$	$2^{1 \times 10^6}$	$2^{6 \times 10^7}$	$2^{3.6 \times 10^9}$	$2^{8.64 \times 10^{10}}$	$2^{2.592 \times 10^{12}}$	$2^{3.1536 \times 10^{13}}$	$2^{3.15576 \times 10^{15}}$
$\sqrt{n}$	1×10^{12}	3.6×10^{15}	1.29×10^{19}	7.46×10^{21}	6.72×10^{24}	9.95×10^{26}	9.96×10^{30}
n	1×10^6	6×10^7	3.6×10^9	8.64×10^{10}	2.59×10^{12}	3.15×10^{13}	3.16×10^{15}
$n \lg n$	62746	2801417	133378058	2755147513	71870856404	797633893349	6.86×10^{13}
n^2	1000	7745	60000	293938	1609968	5615692	56176151
n^3	100	391	1532	4420	13736	31593	146679
2^n	19	25	31	36	41	44	51
$n!$	9	11	12	13	15	16	17

Exercício 1

Expresse a função $n^3/1000 - 100n^2 - 100n + 3$ em termos de notação Θ .

Exercício 2

Faça a mesma análise de complexidade do melhor e pior caso para o algoritmo ordenação por bolha (*bubble sort*).

BUBBLE-SORT(A)

1. **for** $i \leftarrow 1$ **to** $n - 1$
2. **for** $j \leftarrow 1$ **to** $n - i$
3. **if** $A[j] > A[j + 1]$ **then**
4. **do** $swap(A[j], A[j + 1])$

Referências

- CORMEN, T. H.[et al]. Algoritmos: teoria e prática. 3^a ed. Rio de Janeiro: Elsevier, 2012.
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de dados e seus algoritmos. 3. ed. Rio de Janeiro, RJ: LTC, 2010. 302p.